



June 15th, 2026

PENETRATION TEST REPORT:

BRINS BMINE

Technology & Information System Division



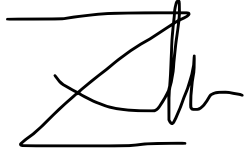


TABLE OF CONTENT

<i>DOCUMENT SIGN-OFF</i>	<i>ii</i>
<i>ASSESSMENT INFORMATION</i>	<i>iii</i>
<i>SCOPING & RULES OF ENGAGEMENT</i>	<i>iv</i>
<i>DOCUMENT METADATA</i>	<i>v</i>
<i>EXECUTIVE SUMMARY OF FINDINGS</i>	<i>1</i>
<i>TECHNICAL FINDINGS</i>	<i>4</i>
1. Finding BMINE-001: Exposed .env File	4
2. Finding BMINE-002: Exposed laravel.log File	6
3. Finding BMINE-003: Exposed .git/* Files	8
4. Finding BMINE-004: Exposed .htaccess File	11
5. Finding BMINE-005: Stored XSS in https://*-app-bmine.brins.co.id/Rekening.....	13
6. Finding BMINE-006: Stored XSS in https://*-app-bmine.brins.co.id/ coveringsubmission/a3m	15
7. Finding BMINE-007: Stored XSS in https://*-app-bmine.brins.co.id/ listviem	17
8. Finding BMINE-008: Stored XSS in https://*-app-bmine.brins.co.id/ upload/covering ..	19
9. Finding BMINE-009: Negative Value Injection in CoveringFlagAsuransi Endpoint	21
10. Finding BMINE-010: Missing Authentication and Authorization on CoveringFlagAsuransi Endpoint.....	23



DOCUMENT SIGN-OFF

TECHNOLOGY & INFORMATION SYSTEM DIVISION	SIGNATURE
Division Head Rully Harisman	
Deputy Division Head Iwan Setiawan	
IT Security Department Head Razin Haq Umran	
Team Member Muhammad Nanda Nugraha	



ASSESSMENT INFORMATION

IT SECURITY DEPARTMENT

Lead Penetration Tester

Razin Haq Umran

Penetration Tester

Muhammad Nanda Nugraha

CLIENT DETAILS

Application Development Department

ASSESSMENT SCOPE SUMMARY

Engagement Timeframe

02/06/2026-08/06/2026

Engagement Scope

Web Application

Project ID: AppDev-Bmine-v1.0-08-06-2026

Report Date: 15-06-2026



SCOPING & RULES OF ENGAGEMENT

Assessment Scope

The predetermined scope for IT Security Department to carry the out the Web Application penetration test was:

Assessment Type

Blackbox

Host

<https://app-bmine.brins.co.id/>



DOCUMENT METADATA

AUTHOR

Muhammad Nanda Nugraha

CO-AUTHOR

Razin Haq Umran

Document Number

SEC-RPT-PNT-2026-08-017





EXECUTIVE SUMMARY OF FINDINGS

IT Security Department conducted a web application penetration test for Application Development Department. This test was performed to assess BMINE web application's vulnerability.

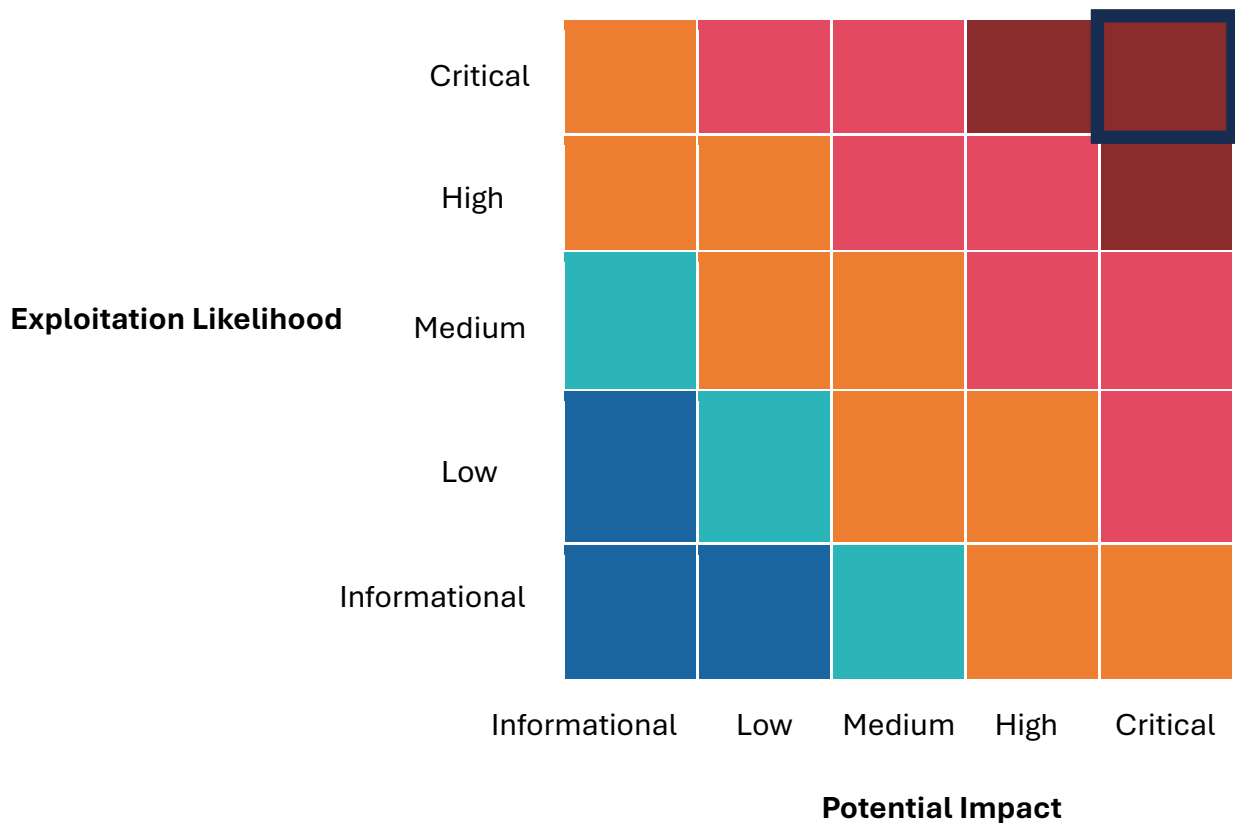
IT Security Department reviewed the security of BMINE web application and determined a **CRITICAL** risk of compromise from any threat actor, as shown by the presence of vulnerability details in this report.

The detailed of findings and remediation recommendations for these assessments may be found later in this report.

BMINE Risk Rating

IT Security Department calculates BMINE web application risk based on Exploitation Likelihood (ease of exploitation) and Potential Impact (potential business impact to the environment).

OVERALL RISK RATING: CRITICAL





Report Card

3	4	1	1	0
Critical	High	Medium	Low	Informational

Findings Summary Table

Finding	Severity	Brief Recommendation
Finding BMINE-001: Exposed .env File	Critical	Exposed .env files present a severe security risk by leaking critical credentials, such as database access details, API keys, and SMTP configurations.
Finding BMINE -002: Exposed laravel.log File	Critical	Unauthorized access to the laravel.log file enables outsiders to view application logs, which can inadvertently disclose system details, error traces, user data, and credentials.
Finding BMINE -003: Exposed .git/* Files	Critical	An exposed .git directory allows malicious actors to download the application's source code and sensitive metadata, drastically increasing the risk of a full system compromise.
Finding BMINE -004: Exposed .htaccess File	Medium	Publicly accessible .htaccess files can reveal internal server configurations, helping attackers understand the active security controls and plan targeted exploits.
Finding BMINE -005: Stored XSS in https://*-app-bmine.brins.co.id/Rekening	High	Enforce Output Encoding: Ensure all user-supplied text data is rendered using secure template syntax (e.g., standard {{ \$variable }} in Laravel Blade) to contextually encode HTML/JavaScript tags. Avoid using unescaped raw HTML formatting ({!! !! })
Finding BMINE -006: Stored XSS in https://*-app-bmine.brins.co.id/coveringsubmission/a3m	High	Enforce Output Encoding: Ensure all user-supplied text data is rendered using secure template syntax (e.g., standard {{ \$variable }} in Laravel Blade) to contextually encode HTML/JavaScript tags. Avoid using unescaped raw HTML formatting ({!! !! })
Finding BMINE -007: Stored XSS in https://*-app-bmine.brins.co.id/listview	High	Enforce Output Encoding: Ensure all user-supplied text data is rendered



		using secure template syntax (e.g., standard {{ \$variable }} in Laravel Blade) to contextually encode HTML/JavaScript tags. Avoid using unescaped raw HTML formatting ({!! !! })
Finding BMINE -008: Stored XSS in https://*-app-bmine.brins.co.id/upload/covering	High	Enforce Output Encoding: Ensure all user-supplied text data is rendered using secure template syntax (e.g., standard {{ \$variable }} in Laravel Blade) to contextually encode HTML/JavaScript tags. Avoid using unescaped raw HTML formatting ({!! !! })
Finding BMINE -009 Can negative in endpoint https://dev-brisurf.brins.co.id/bmine/CoveringFlagAsuransi	Medium	Apply Range Validation: Implement numeric range boundaries on the server-side validation layer to ensure financial and premium parameters cannot accept values less than zero (e.g., 'OriginalPremi' => 'required numeric min:0').
Finding BMINE -010: Missing Authentication and Authorization on CoveringFlagAsuransi Endpoint	High	Enforce Authentication & Authorization Middleware: Restrict access to the CoveringFlagAsuransi endpoint by implementing a secure API gateway mechanism (such as Laravel Sanctum, Passport, or Bearer JWT tokens).

Summary of Strengths

The application adopts several security best practices, including the use of environment variables to separate sensitive configuration data from source code. Additionally, the application structure and deployment model indicate an awareness of secure development principles, helping to reduce the exposure of sensitive information when properly configured.

Summary of Weaknesses

Multiple sensitive files and resources were found to be publicly accessible, including the .env file, .git repository, and laravel.log file. These exposures may allow attackers to obtain source code, application logs, credentials, API keys, database connection details, and other sensitive information. Such information could be leveraged to gain unauthorized access to internal systems, compromise the confidentiality and integrity of data, abuse third-party services, identify additional vulnerabilities, and facilitate further attacks against the application and its supporting infrastructure.

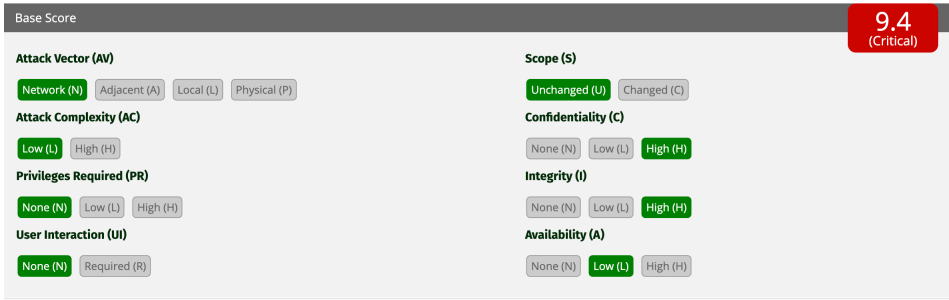
Strategic Recommendation

Restrict public access to sensitive files (.env, .git, logs) through proper server configuration. Rotate any exposed credentials or secrets and perform regular security assessments and secret scanning to prevent future exposure.



TECHNICAL FINDINGS

1. Finding BMINE-001: Exposed .env File

Description:	This vulnerability arises due to security misconfiguration on the web server or the application's directory settings. As a result, sensitive configuration files (.env) that should be hidden can instead be freely accessed by the public through a browser.
Risk:	Database Leaks and Manipulation – The attacker got the host, username, and database password. If the database port is open to the public, the attacker can just log in to steal user data. Email Hijacking – Exposed SMTP credentials can be abused to send phishing, spam, or spoofed emails. API and Cloud Credential Exposure – Leaked API keys and cloud credentials may enable unauthorized access, resulting in data theft, service abuse, or financial loss. Malware Injection and Exploitation – Attackers may leverage exposed configurations to inject malware, modify system settings, or conduct further attacks. 
System:	Endpoint: https://app-bmine.brins.co.id/.env
Tools Used:	cURL
References:	OWASP A05:2021: Security Misconfiguration
Remediation:	Ensure the Laravel application is configured with the web server document root pointing to the public directory and restrict direct access to the .env file. Rotate all exposed credentials, including database, SMTP, API, and cloud service secrets. Consider using a dedicated secret management solution and regularly review server configurations to prevent sensitive file exposure.



Reproduce Steps:

1. Access the .env file directly via web browser, <https://app-bmine.brins.co.id/.env>. The file should not be publicly accessible.

Proof of Concept:

```
env (8)
APP_NAME="BMINE_UI"
APP_ENV=production
APP_KEY="..."
APP_DEBUG=false
APP_URL=https://app-bmine.brins.co.id
#APP_URL=http://192.168.1.1
APP_SUBDOMAIN_URL=https://app-bmine.brins.co.id
ASSET_URL=https://app-bmine.brins.co.id
APP_IPADDR=192.168.1.1
APP_VERSION=1.1.0
APP_FORCE_HTTPS=true

#LOG_CHANNEL=stack
LOG_CHANNEL=single

DB_CONNECTION=mysqlsrv
DB_HOST=192.168.1.1
DB_PORT=1433
DB_DATABASE=BMINE
DB_USERNAME=sa
DB_PASSWORD=Bm1n3DB2022

DB_CONNECTION=mysqlsrv
DB_HOST_DATA=192.168.1.1
DB_PORT_DATA=1433
DB_DATABASE_DATA=PROGRAM_DB
DB_USERNAME_DATA=sa
DB_PASSWORD_DATA=Bm1n3DB2022

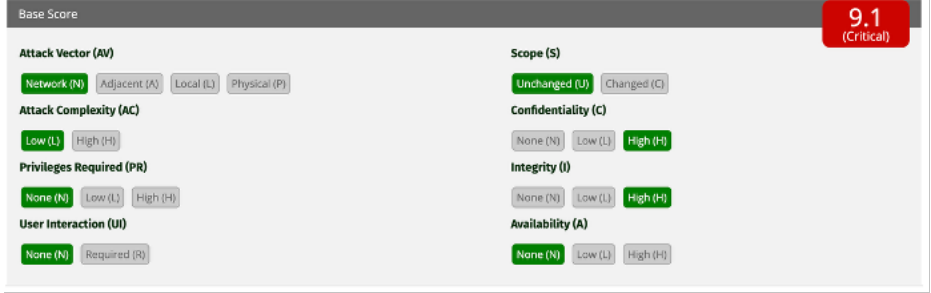
DB_CONNECTION=mysql
DB_HOST_DATA_PAYMENT=192.168.1.1
DB_PORT_DATA_PAYMENT=1433
DB_DATABASE_DATA_PAYMENT=PAYMENT_DB
DB_USERNAME_DATA_PAYMENT=
DB_PASSWORD_DATA_PAYMENT=Bm1r

DB_CONNECTION=mysqlsrv
DB_HOST_DATA_COVERING=192.168.1.1
DB_PORT_DATA_COVERING=1433
DB_DATABASE_DATA_COVERING=COVERING
DB_USERNAME_DATA_COVERING=sa
DB_PASSWORD_DATA_COVERING=Bm1n3DB2022

DB_CONNECTION=mysql
DB_HOST_DATA_REPORT=192.168.1.1
DB_PORT_DATA_REPORT=1433
DB_DATABASE_DATA_REPORT=REPORT_DB
DB_USERNAME_DATA_REPORT=sa
DB_PASSWORD_DATA_REPORT=Bm1n3DB2022
```



2. Finding BMINE-002: Exposed laravel.log File

Description:	An exposed laravel.log file allows unauthorized users to view application logs, potentially disclosing sensitive information such as system details, error messages, user data, and credentials.
Risk:	Information Disclosure – Application logs may contain sensitive information such as system paths, stack traces, usernames, email addresses, session identifiers, API keys, or configuration details. Credential Exposure – Error logs may inadvertently reveal credentials or authentication tokens that could be abused for unauthorized access. Attack Facilitation – Detailed error messages and application behavior information can help attackers identify vulnerabilities and develop targeted attacks against the application. Privacy and Compliance Impact – Exposure of user-related information through logs may result in privacy violations and potential regulatory compliance issues.  The image shows a Base Score of 9.1 (Critical). The settings for the score are: Attack Vector (AV) set to Network (N), Attack Complexity (AC) set to Low (L), Privileges Required (PR) set to None (N), User Interaction (UI) set to None (N), Scope (S) set to Unchanged (U), Confidentiality (C) set to High (H), Integrity (I) set to High (H), and Availability (A) set to None (N).
System:	Endpoint: https://app-bmine.brins.co.id/storage/logs/laravel.log
Tools Used:	cURL
References:	OWASP A05:2021: Security Misconfiguration https://hacktricks.wiki/en/network-services-pentesting/pentesting-web/laravel.html
Remediation:	Restrict public access to the laravel.log file, store logs outside the web root, and configure the web server to deny direct access to log files. Additionally, review and sanitize sensitive information contained within existing logs.



Reproduce Steps:

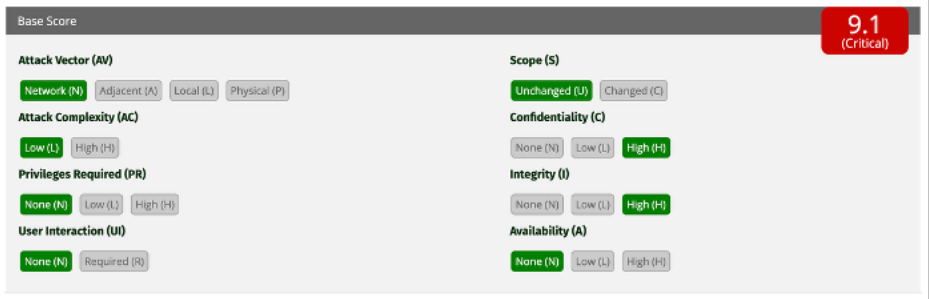
1. Access the Laravel.log file the directly via web browser, <https://app-brmine.brins.co.id/storage/logs/laravel.log>. The file should not be publicly accessible.

Proof of Concept:

```
\Request))
#43 /var/www/html/vendor/laravel/framework/src/Illuminate/Foundation/Http/Kernel.php(142): Illuminate\Pipeline\Pipeline->then(Object(Closure))
#44 /var/www/html/vendor/laravel/framework/src/Illuminate/Foundation/Http/Kernel.php(111): Illuminate\Foundation\Http\Kernel->sendRequestThroughRouter(Object(Illuminate\Http
\Request))
#45 /var/www/html/index.php(52): Illuminate\Foundation\Http\Kernel->handle(Object(Illuminate\Http\Request))
#46 {main}
}
[2024-10-30 22:41:37] laravel.EMERGENCY: Unable to create configured logger. Using emergency logger. {"exception":"[object] (MongoDB\\Driver\\Exception\\
InvalidArgument\\Exception(code: 0): Failed to parse MongoDB URI: 'mongodb://homestead:secret@127.0.0.1:27017?authSource=homestead/'. Invalid host string in URI. at /var/www/html/
vendor/mongodb/mongodb/src/Client.php:124)
[stacktrace]
#0 /var/www/html/vendor/mongodb/mongodb/src/Client.php(124): MongoDB\\Driver\\Manager->__construct('mongodb://homestead:secret@127.0.0.1:27017?authSource=homestead', Array, Array)
#1 /var/www/html/app/Logging/Mongologger.php(21): MongoDB\\Client->__construct('mongodb://homestead:secret@127.0.0.1:27017?authSource=homestead', Array, Array)
#2 /var/www/html/vendor/laravel/framework/src/Illuminate/Log/LogManager.php(237): App\\Logging\\Mongologger->__invoke(Array)
#3 /var/www/html/vendor/laravel/framework/src/Illuminate/Log/LogManager.php(210): Illuminate\\Log\\LogManager->createCustomDriver(Array)
#4 /var/www/html/vendor/laravel/framework/src/Illuminate/Log/LogManager.php(125): Illuminate\\Log\\LogManager->resolve('mongologger', Array)
#5 /var/www/html/vendor/laravel/framework/src/Illuminate/Log/LogManager.php(112): Illuminate\\Log\\LogManager->get('mongologger')
#6 /var/www/html/vendor/laravel/framework/src/Illuminate/Log/LogManager.php(101): Illuminate\\Log\\LogManager->driver('mongologger')
#7 /var/www/html/vendor/laravel/framework/src/Illuminate/Log/LogManager.php(261): Illuminate\\Log\\LogManager->channel('mongologger')
#8 [internal function]: Illuminate\\Log\\LogManager->Illuminate\\Log\\Closure('mongologger', 0)
#9 /var/www/html/vendor/laravel/framework/src/Illuminate/Collections/Collection.php(695): array_map(Object(Closure), Array, Array)
#10 /var/www/html/vendor/laravel/framework/src/Illuminate/Collections/Traits/EnumeratesValues.php(354): Illuminate\\Support\\Collection->map(Object(Closure))
#11 /var/www/html/vendor/laravel/framework/src/Illuminate/Log/LogManager.php(262): Illuminate\\Support\\Collection->flatMap(Object(Closure))
#12 /var/www/html/vendor/laravel/framework/src/Illuminate/Log/LogManager.php(210): Illuminate\\Log\\LogManager->createStackDriver(Array)
#13 /var/www/html/vendor/laravel/framework/src/Illuminate/Log/LogManager.php(125): Illuminate\\Log\\LogManager->resolve('stack', Array)
#14 /var/www/html/vendor/laravel/framework/src/Illuminate/Log/LogManager.php(112): Illuminate\\Log\\LogManager->get('stack')
#15 /var/www/html/vendor/laravel/framework/src/Illuminate/Log/LogManager.php(590): Illuminate\\Log\\LogManager->driver()
#16 /var/www/html/vendor/laravel/framework/src/Illuminate/Foundation/Exceptions/Handler.php(252): Illuminate\\Log\\LogManager->error('No application ...', Array)
#17 /var/www/html/vendor/laravel/framework/src/Illuminate/Routing/Pipeline.php(49): Illuminate\Foundation\\Exceptions\\Handler->report(Object(Illuminate\\Encryption\\
MissingAppKeyException))
#18 /var/www/html/vendor/laravel/framework/src/Illuminate/Pipeline/Pipeline.php(172): Illuminate\\Routing\\Pipeline->handleException(Object(Illuminate\\Http\\Request),
```



3. Finding BMINE-003: Exposed .git/* Files

Description:	An exposed .git directory may allow attackers to access the application's source code and sensitive information, increasing the risk of unauthorized access and further compromise of the environment.
Risk:	Source Code Disclosure – Attackers may obtain the application's source code, enabling them to identify vulnerabilities, business logic flaws, and security weaknesses. Sensitive Information Exposure – The repository may contain credentials, API keys, configuration files, or other sensitive information that could facilitate unauthorized access. Attack Facilitation – Access to the internal repository can provide valuable intelligence about the application's architecture and implementation, increasing the likelihood of successful attacks against the application and its supporting infrastructure. 
System:	Endpoint: https://app-bmine.brins.co.id/.git/index https://app-bmine.brins.co.id/.git/config https://app-bmine.brins.co.id/.git/head
Tools Used:	cURL
References:	OWASP A05:2021 – Security Misconfiguration https://hacktricks.wiki/en/network-services-pentesting/pentesting-web/laravel.html
Remediation:	Restrict public access to the .git/* files, store logs outside the web root, and configure the web server to deny direct access to log files.



Reproduce Steps:

1. Access all the associated files as mentioned above directly via web browser. Those file should not be publicly accessible

Proof of Concept:

<https://app-bmine.brins.co.id/.git/index>:

```
clean_structure2.txt
38 -resources/views/coveringcancel/view.blade.php
39 -resources/views/coveringreject/view.blade.php
40 -resources/views/dashboard/a3m/index.blade.php
41 -resources/views/viewdetail/listview.blade.php
42 .app/Console/Commands/GenerateReportCommand.php
43 .app/Exports/DataListNullCoveringFlagExport.php
44 .app/Http/Controllers/BrilinkAgenController.php
45 .app/Http/Controllers/CovFlagAkuaController.php
46 .app/Http/Controllers/CovSubAssetController.php
47 .app/Http/Controllers/CovValAsmikController.php
48 .app/Http/Controllers/ProgramcodeController.php
49 .app/Models/coveringapproval/MS_StatusModel.php
50 .env.example
51 .env.prod
52 .gitlab-ci.yml
53 .public/LandingPage/vendor/jquery/jquery.min.js
54 .public/LandingPage/vendor/owl.carousel
55 .public/assets/js/page/gmaps-multiple-marker.js
56 .public/assets/modules/chocolat/dist/index.html
```

<https://app-bmine.brins.co.id/config>:

```
config (4)
[core]
    repositoryformatversion = 0
    filemode = 040000
    bare = false
    logallrefupdates = true
[branch "production"]
    remote = origin
    merge = refs/heads/production
[branch "35-hot-fix-perbaikan-controller-reporting"]
    remote = origin
    merge = refs/heads/35-hot-fix-perbaikan-controller-reporting
[user]
    email = k6ba9ew81fkdz...@org
[remote "origin"]
    url = https://.../PA9rEYpX4RjgQffp1hiuv...
    fetch = +refs/heads/*:refs/heads/*
[branch "hotfix-bot-telegram"]
    remote = origin
    merge = refs/heads/hotfix-bot-telegram
```

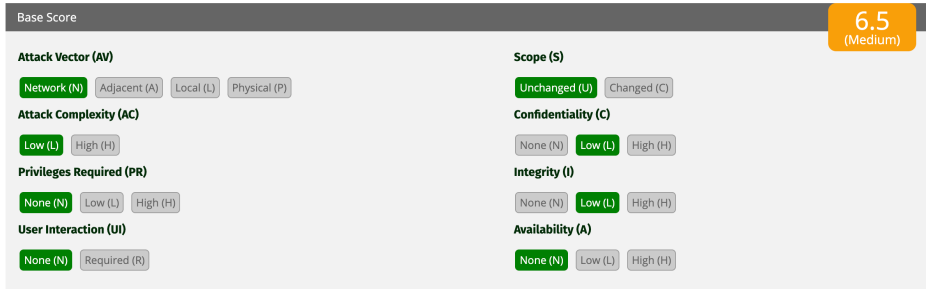


<https://app-bmine.brins.co.id/.git/head:>

HEAD (1)
ref: refs/heads/production



4. Finding BMINE-004: Exposed .htaccess File

Description:	An exposed .htaccess file may reveal sensitive server configuration details, assisting attackers in understanding the application's security controls and facilitating further attacks against the environment..
Risk:	Information Disclosure – Attackers may gain insight into server configurations, directory structures, URL rewriting rules, and security controls. Attack Facilitation – Exposed configuration details can help attackers identify weaknesses and develop more targeted attacks.  The image shows a Base Score of 6.5 (Medium). The settings for the score are: Attack Vector (AV) set to Network (N), Attack Complexity (AC) set to Low (L), Privileges Required (PR) set to None (N), User Interaction (UI) set to None (N), Scope (S) set to Unchanged (U), Confidentiality (C) set to Low (L), Integrity (I) set to Low (L), and Availability (A) set to None (N).
System:	Endpoint: https://app-bmine.brins.co.id/.htaccess
Tools Used:	cURL
References:	OWASP A05:2021: Security Misconfiguration https://hacktricks.wiki/en/network-services-pentesting/pentesting-web/laravel.html
Remediation:	Restrict public access to the .htaccess file and ensure sensitive configuration files cannot be accessed directly through the web server. Review the file for sensitive information and implement appropriate access controls to prevent unauthorized disclosure.



Reproduce Steps:

1. Access the .htaccess file via web browser, <https://app-bmine.brins.co.id/.htaccess>. The file should not be publicly accessible.

Proof of Concept:

```
htaccess (4)
<IfModule mod_rewrite.c>
  <IfModule mod_negotiation.c>
    Options -MultiViews -Indexes
  </IfModule>

  RewriteEngine On

  # Handle Authorization Header
  RewriteCond %{HTTP:Authorization} .
  RewriteRule .* - [E=HTTP_AUTHORIZATION:%{HTTP:Authorization}]

  # Redirect Trailing Slashes If Not A Folder...
  RewriteCond %{REQUEST_FILENAME} !-d
  RewriteCond %{REQUEST_URI} (.+)/$
  RewriteRule ^ %1 [L,R=301]

  # Send Requests To Front Controller...
  RewriteCond %{REQUEST_FILENAME} !-d
  RewriteCond %{REQUEST_FILENAME} !-f
  RewriteRule ^ index.php [L]
</IfModule>
```



5. Finding BMINE-005: Stored XSS in https://*-app-bmine.brins.co.id/Rekening

Description:	A Stored Cross-Site Scripting (Stored XSS) vulnerability was identified in the web application at app-bmine.brins.co.id/Rekening. This security flaw occurs because the application fails to perform sufficient input validation or special character sanitization on the "Add New Account" (Penambahan Data Rekening Baru) form.
Risk:	Account Takeover (Session Hijacking): Attackers can steal session cookies or authentication tokens via JavaScript, allowing them to impersonate users or administrators without knowing their passwords. Data Theft & Manipulation: The stored malicious script can harvest sensitive information displayed on the page or alter critical account data on the fly (e.g., modifying bank account details visually). Malicious Redirection: The browser can be forced to redirect users to external phishing websites designed to steal credentials. Widespread Exposure (Persistence): Since the payload is permanently stored in the database, every single user or administrator who visits the /Rekening page will automatically execute the malicious script, magnifying the attack scale.
System:	https://*-app-bmine.brins.co.id/Rekening
Tools Used:	Burp Suite
References:	OWASP TOP 10: A03:2021-Injection.
Remediation:	Context-Aware Output Encoding (Primary Defense): Convert special characters into safe HTML entities (e.g., convert < to &lt; and > to &gt;) before displaying data from the database onto the page. This ensures the browser treats the input as plain text, not executable code. Use Framework Auto-Escaping: Leverage built-in security mechanisms of modern frameworks (such as Laravel's {{ \$variable }} or React/Angular templates) and avoid raw output methods (like {!! \$variable !!}). Input Sanitization: Apply robust filtering using trusted libraries like <i>DOMPurify</i> or <i>HTMLPurify</i> to strip out dangerous tags (<script>, onload, onerror) before saving the data to the database. Implement HttpOnly Flag: Set the HttpOnly flag on all session cookies to prevent malicious JavaScript from accessing or stealing them, effectively mitigating the risk of session hijacking. Enforce Content Security Policy (CSP): Implement a strict CSP HTTP header to restrict the execution of unauthorized or inline scripts within the application



Reproduce Steps:

1. Click the "Insert Master Rekening" button.
2. Fill in the available field with the following XSS payload: `<script>alert(1);</script>`
3. Click the "Save" button to submit the data..

Proof of Concept:

The screenshot displays the 'List Master Rekening' interface. At the top, there's a notification box stating 'dev-app-bmine.brins.co.id says 1'. Below this, the 'Filtering' section contains two dropdown menus: 'Cabang/MRO' (set to 'Pilih Cabang/MRO') and 'Status Approved' (set to '-- Pilih Status Approved --'). A 'Proses' button is located below the filters. The 'Daftar List Master Rekening' section shows a table with 10 entries. The table columns are: Kode Cabang, Nama Cabang, No Rekening, Nama Rekening, Nama Bank, Status Rekening, Status Approval, and Aksi. The first four rows are visible, showing various branches and their associated account details. A red speech bubble icon is positioned over the 'Detail' link in the first row of the table.

Kode Cabang	Nama Cabang	No Rekening	Nama Rekening	Nama Bank	Status Rekening	Status Approval	Aksi
115	KANTOR CABANG UTAMA	034101001224302	PT. BRI ASURANSI INDONESIA	BRI	Sudah Aktif	Sudah DiSetujui	Detail
100	KANTOR PUSAT	011001001111111	PT. XYZ	BRI	Sudah Aktif	Sudah DiSetujui	Detail
100	KANTOR PUSAT	011001001111112	PT. ABC	BRI	Sudah Aktif	Sudah DiSetujui	Detail
100	KANTOR PUSAT	034101001224302	PT. BRI ASURANSI 4123	BRINS 4123	Sudah Aktif	Sudah DiSetujui	Detail



6. Finding BMINE-006: Stored XSS in https://*-app-bmine.brins.co.id/coveringsubmission/a3m

Description:	A Stored Cross-Site Scripting (XSS) vulnerability was identified in the insurance flag configuration module. The application fails to properly sanitize or contextually encode the user-supplied input for the CoveringFlagAsuransi field before storing it in the database. As a result, when this data is rendered back on the interface, arbitrary JavaScript code can be executed within the context of the user's browser session.
Risk:	Account Takeover (Session Hijacking): Attackers can steal session cookies or authentication tokens via JavaScript, allowing them to impersonate users or administrators without knowing their passwords. Data Theft & Manipulation: The stored malicious script can harvest sensitive information displayed on the page or alter critical account data on the fly (e.g., modifying bank account details visually). Malicious Redirection: The browser can be forced to redirect users to external phishing websites designed to steal credentials. Widespread Exposure (Persistence): Since the payload is permanently stored in the database, every single user or administrator who visits the /Rekening page will automatically execute the malicious script, magnifying the attack scale.
System:	https://*-app-bmine.brins.co.id/coveringsubmission/a3m
Tools Used:	Burp Suite
References:	OWASP TOP 10: A03:2021-Injection.
Remediation:	Context-Aware Output Encoding (Primary Defense): Convert special characters into safe HTML entities (e.g., convert < to &lt; and > to &gt;) before displaying data from the database onto the page. This ensures the browser treats the input as plain text, not executable code. Use Framework Auto-Escaping: Leverage built-in security mechanisms of modern frameworks (such as Laravel's {{ \$variable }} or React/Angular templates) and avoid raw output methods (like {!! \$variable !!}). Input Sanitization: Apply robust filtering using trusted libraries like <i>DOMPurify</i> or <i>HTMLPurify</i> to strip out dangerous tags (<script>, onload, onerror) before saving the data to the database. Implement HttpOnly Flag: Set the HttpOnly flag on all session cookies to prevent malicious JavaScript from accessing or stealing them, effectively mitigating the risk of session hijacking. Enforce Content Security Policy (CSP): Implement a strict CSP HTTP header to restrict the execution of unauthorized or inline scripts within the application



Reproduce Steps:

1. Access the target endpoint at: <https://dev-brisurf.brins.co.id/bmine/CoveringFlagAsuransi>
2. Locate the input field intended for the name or other string parameters on the page.
3. Inject the following Cross-Site Scripting (XSS) test payload into that field: `<script>alert(1);</script>`
4. Click the **Send** (or Submit) button to process the request.
5. Verify that the script executes (e.g., a browser alert window appears) when the page reloads or when the saved data is rendered.

Proof of Concept:

The screenshot shows a web application interface for BRI Insurance. A browser alert box is visible, displaying the message "dev-app-bmine.brins.co.id says 1". The page title is "List Covering Submission". The interface includes a sidebar with navigation links: Dashboard, Admin, Master, Covering, Claim, Reporting, Pembayaran, Proses, and Reconcile. The main content area has a "Filtering Data" section with several input fields for filtering data, including "FID Program", "Nomor Rekening Pinjaman", "Nama Peserta", "Nomor Peserta", "Batch", "Tanggal Transaksi Awal", and "Tanggal Transaksi Akhir". Below the filters are "Search" and "Reset Filter" buttons. At the bottom, there is a table with columns: Nama Program, Channel, Batch, Nama Peserta, Nomor Peserta, Nomor Polis, Nomor Rekening Pinjaman, and Cabang BR. The table contains two rows of data, with the first row showing "9927 | (('ON'))E", "BSPR | BSPR", "210", "@((ON))E", "0000M.00119.2024.01.3463.1.9", "-", "021201502001511", and "0119 | KC Maumere".



7. Finding BMINE-007: Stored XSS in https://*-app-bmine.brins.co.id/listviem

Description:	A Stored Cross-Site Scripting (XSS) vulnerability was identified in the insurance flag configuration module. The application fails to properly sanitize or contextually encode the user-supplied input for the CoveringFlagAsuransi field before storing it in the database. As a result, when this data is rendered back on the interface, arbitrary JavaScript code can be executed within the context of the user's browser session.
Risk:	Account Takeover (Session Hijacking): Attackers can steal session cookies or authentication tokens via JavaScript, allowing them to impersonate users or administrators without knowing their passwords. Data Theft & Manipulation: The stored malicious script can harvest sensitive information displayed on the page or alter critical account data on the fly (e.g., modifying bank account details visually). Malicious Redirection: The browser can be forced to redirect users to external phishing websites designed to steal credentials. Widespread Exposure (Persistence): Since the payload is permanently stored in the database, every single user or administrator who visits the /Rekening page will automatically execute the malicious script, magnifying the attack scale.
System:	https://*-app-bmine.brins.co.id/listview
Tools Used:	Burp Suite
References:	OWASP TOP 10: A03:2021-Injection.
Remediation:	Context-Aware Output Encoding (Primary Defense): Convert special characters into safe HTML entities (e.g., convert < to &lt; and > to &gt;) before displaying data from the database onto the page. This ensures the browser treats the input as plain text, not executable code. Use Framework Auto-Escaping: Leverage built-in security mechanisms of modern frameworks (such as Laravel's {{ \$variable }} or React/Angular templates) and avoid raw output methods (like {!! \$variable !!}). Input Sanitization: Apply robust filtering using trusted libraries like <i>DOMPurify</i> or <i>HTMLPurify</i> to strip out dangerous tags (<script>, onload, onerror) before saving the data to the database. Implement HttpOnly Flag: Set the HttpOnly flag on all session cookies to prevent malicious JavaScript from accessing or stealing them, effectively mitigating the risk of session hijacking. Enforce Content Security Policy (CSP): Implement a strict CSP HTTP header to restrict the execution of unauthorized or inline scripts within the application



Reproduce Steps:

1. Access the page https://*-app-bmine.brins.co.id/listview.
2. Click on the Tambah List View Detail menu.
3. Inject the payload `<script>alert(1);</script>` into the vulnerable parameter or input form.
4. Click Tambah to submit the form and execute the payload.

Proof of Concept:

dev-app-bmine.brins.co.id says 1

OK

Show 10 entries

Jenis Asuransi	Nama Kolom	Alias Kolom	Tipe Data	Kategori	Urutan	Status	Aksi
Asuransi Kredit 3 Miliar	FIDJenisAgunan	Jenis Agunan	-	Covering_A3M		Aktif	 
Asuransi Kredit 3 Miliar	FIDAgunan	FID Agunan	-	Covering_A3M		Aktif	 
Asuransi Kredit 3 Miliar	KodePosAgunan	Kode Pos Agunan	-	Covering_A3M		Aktif	 
Asuransi Kredit 3 Miliar	AlamatAgunan	Alamat Agunan	-	Covering_A3M		Aktif	 
Asuransi Kredit 3 Miliar	KodeOkupasi	Kode Okupasi	-	Covering_A3M		Aktif	 
Asuransi Kredit 3 Miliar	FlagPasar	Flag Pasar	-	Covering_A3M		Aktif	 
Asuransi Kredit 3 Miliar	KodePasar	Kode Pasar	-	Covering_A3M		Aktif	 
Asuransi Kredit 3 Miliar	RangkaBangunan	Rangka Bangunan	-	Covering_A3M		Aktif	 
Asuransi Kredit 3 Miliar	PenutupAtap	Penutup Atap	-	Covering_A3M		Aktif	 
Asuransi Kredit 3 Miliar	BahanLantai	Bahan Lantai	-	Covering_A3M		Aktif	 

Showing 31 to 40 of 516 entries

Previous 1 2 3 4 5 ... 52 Next

PT BRI ASURANSI INDONESIA
Jl.Mampang Prapatan Raya No.18 Jakarta Selatan - 12790 Indonesia
Telp +62 21-791-704-7778 | Fax:+62 21 791-905-14.791-974-32 | Email:info@brins.co.id | Website:www.brins.co.id
Call Center: +62(21) 1500699
Copyright © TSI BRI Insurance 2021 All rights reserved.



8. Finding BMINE-008: Stored XSS in https://*-app-bmine.brins.co.id/upload/covering

Description:	A Stored Cross-Site Scripting (XSS) vulnerability was identified in the insurance flag configuration module. The application fails to properly sanitize or contextually encode the user-supplied input for the Upload Covering field before storing it in the database. As a result, when this data is rendered back on the interface, arbitrary JavaScript code can be executed within the context of the user's browser session.
Risk:	Account Takeover (Session Hijacking): Attackers can steal session cookies or authentication tokens via JavaScript, allowing them to impersonate users or administrators without knowing their passwords. Data Theft & Manipulation: The stored malicious script can harvest sensitive information displayed on the page or alter critical account data on the fly (e.g., modifying bank account details visually). Malicious Redirection: The browser can be forced to redirect users to external phishing websites designed to steal credentials. Widespread Exposure (Persistence): Since the payload is permanently stored in the database, every single user or administrator who visits the /Rekening page will automatically execute the malicious script, magnifying the attack scale.
System:	https://*-app-bmine.brins.co.id/upload/covering
Tools Used:	Burp Suite
References:	OWASP TOP 10: A03:2021-Injection.
Remediation:	Context-Aware Output Encoding (Primary Defense): Convert special characters into safe HTML entities (e.g., convert < to &lt; and > to &gt;) before displaying data from the database onto the page. This ensures the browser treats the input as plain text, not executable code. Use Framework Auto-Escaping: Leverage built-in security mechanisms of modern frameworks (such as Laravel's {{ \$variable }} or React/Angular templates) and avoid raw output methods (like {!! \$variable !!}). Input Sanitization: Apply robust filtering using trusted libraries like <i>DOMPurify</i> or <i>HTMLPurify</i> to strip out dangerous tags (<script>, onload, onerror) before saving the data to the database. Implement HttpOnly Flag: Set the HttpOnly flag on all session cookies to prevent malicious JavaScript from accessing or stealing them, effectively mitigating the risk of session hijacking. Enforce Content Security Policy (CSP): Implement a strict CSP HTTP header to restrict the execution of unauthorized or inline scripts within the application



Reproduce Steps:

1. Download the provided Excel template from the [Insert Feature/Page Name] section..
2. Open the downloaded Excel file and inject the following XSS payload into one of the data fields (e.g., 'Name' or 'Description' column):
`<script>alert(1);</script>`.
3. Save the file and upload the modified Excel document back to the application.
4. Click the "Upload" (or "Submit") button to send the file to the server.
5. Navigate to the page where the uploaded Excel data is rendered/displayed, and observe that the XSS payload triggers and executes successfully in the browser.

Proof of Concept:

The screenshot displays a web application interface for 'BRI insurance'. The main content area is titled 'Upload Manual Covering' and contains a section for 'Upload Covering'. It includes a 'Step 1: Download Template (Opsional)' and a list of 'Informasi Format dan Pengisian Kolom'. The bottom of the image shows a network request log for 'GET upload/covering' with a status of 200 and various headers.



9. Finding BMINE-009: Negative Value Injection in CoveringFlagAsuransi Endpoint

Description:	During the security assessment, a business logic vulnerability was identified on the endpoint https://dev-brisurf.brins.co.id/bmine/CoveringFlagAsuransi. The backend application fails to validate the integrity and range of numeric input fields within the payload data structure. Specifically, parameters representing financial calculations, premiums, or insurance coverage limits—such as OriginalPremi, NominalPremi, Plafon, or related calculation flags—can be manipulated by injecting negative values (e.g., -7000000.00). Because the application does not implement strict non-negative server-side validation, these negative inputs are accepted, processed, and potentially saved into the backend database.
Risk:	Financial and Premium Manipulation: Attackers can inject negative values into premium-related fields to arbitrarily lower the total calculated cost, bypass payment thresholds, or potentially generate a negative balance/refund state within the system. Data and Ledger Corruption: Allowing negative values in fields designed strictly for positive integers or decimals disrupts the integrity of financial accounting, reporting ledgers, and insurance coverage statistics. Downstream Logic Failure (Type Confusion/Arithmetic Errors): Submitting unexpected negative values can break downstream automated workflows, such as batch processing, broker commission calculations (NominalKomisiTotal), or tax evaluations, causing unhandled application exceptions or system crashes.
System:	Bmine CoveringFlagAsuransi
Tools Used:	Burp suite
References:	OWASP: A04:2021 – Insecure Design.
Remediation:	Do not rely on client-side constraints. Implement strict server-side validation rules using Laravel's Form Request Validation to ensure that all financial and premium parameters are strictly greater than or equal to zero.



Reproduce Steps:

1. Modify the JSON payload parameter values by injecting a negative sign (-) into financial or premium fields.

Example :

```
{  
  "FIDProgram": 9927,  
  "NamaProgram": "ONE",  
  "Plafon": "2000000000",  
  "OriginalPremi": "-7000000.00",  
  "NominalPremi": "-14000000",  
  "NominalKomisiTotal": "852638.00"  
  // ... [rest of the JSON payload]  
}
```

2. Send the manipulated request through your API client or forward it via your proxy tool.

Proof of Concept:

Tanggal Mulai Covering : 01/01/2023
Tanggal Akhir Covering : 07/12/2026
Dokumen Kepesertaan [Unduh](#)

Data Premi

Original Premi	Rp. -7,000,000.00
Disakon Premi	Rp. 0.00
Nominal Pertanggungan	Rp. 2,000,000,000.00
Nominal Coverage	Rp. 1,462,500,000.00
Nominal Fee Covering	Rp. 0.00
Nominal Pajak Covering	Rp. 0.00
Persentase Komisi Broker	: 7.5000 %
Nominal Komisi	Rp. 0.00
Persentase Pertanggungan	: 1.0000 %
Persentase Coverage	: 0.7500 %
Perhitungan Premi	: 0.7% x (12 / 12) x 2,000,000,000 = Rp.14,000,000.00
Teller ID	: 0013892
Tanggal Original Teller Premium	: 19/12/2022
Tanggal Teller Premium	: 19/12/2022
Tanggal Original Teller Fee	: 19/12/2022
Tanggal Teller Fee	: 19/12/2022
Key Asuransi Rekanan	: CNMAJ
Nama Asuransi Rekanan	: BRI Insurance
Perhitungan Jangka Waktu	: 46
Nominal Biaya Admin	Rp. 0.00
Additional Premi	Rp. 0.00
Rate Premi	: 0.7000
Tanggal Sukses Covering	: 01/08/2024
FID Jenis Pertanggungan	: 3
Remark Premi	: BRISURF_COV_0000M.2024.01.3463.1.8



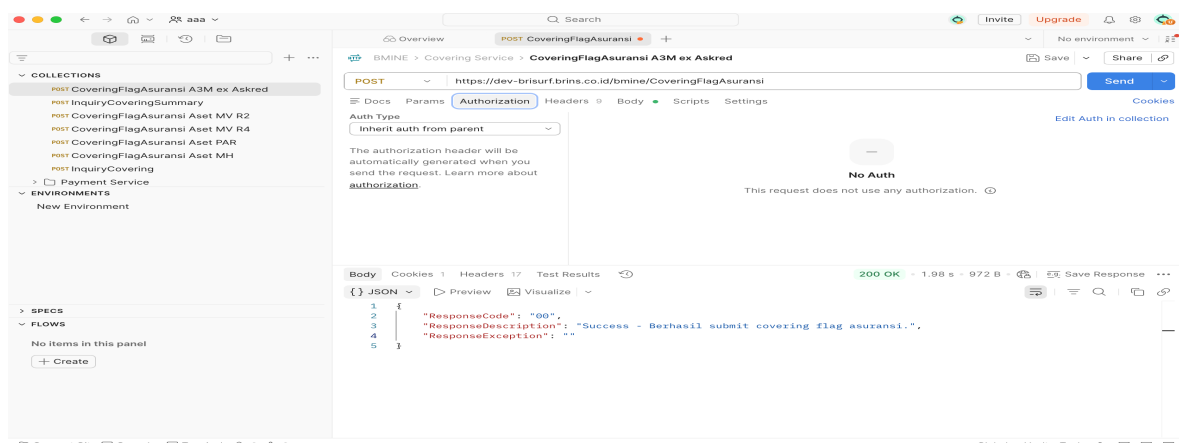
10. Finding BMINE-010: Missing Authentication and Authorization on CoveringFlagAsuransi Endpoint

Description:	A critical security vulnerability was identified on the CoveringFlagAsuransi API endpoint. The application currently fails to enforce any Authentication (verifying the identity of the user) or Authorization (verifying the permissions of the user) mechanisms on this route. Consequently, the endpoint is entirely exposed to the public internet. Any unauthenticated external actor can construct and send arbitrary HTTP requests (such as POST or PUT payloads) directly to this API gateway. The server will accept, process, and permanently commit the data to the backend database without validating whether the request originated from a legitimate, logged-in user or an authorized system process.
Risk:	Total API Abuse: External unauthenticated actors can send raw requests directly to the API, allowing them to create arbitrary insurance entries, perform mass data scraping, or execute bulk record manipulation without any log accountability
System:	https://dev-brisurf.brins.co.id/bmine/CoveringFlagAsuransi
Tools Used:	Burp suite
References:	OWASP: A04:2021 – Insecure Design.
Remediation:	Implement Middleware Security Gateways: Secure the API route immediately behind a hardened authentication middleware cluster (e.g., Laravel Sanctum, Passport, or centralized Bearer JWT authenticators). Enforce Role-Based Privileges (RBAC): Map endpoint route access to specific organizational permissions. Ensure that incoming authenticated connections are programmatically verified against strict access policies before processing read/write operations.

Reproduce Steps:

1. **Open** Postman or terminal (cURL). Do **not** include any login credentials, session cookies, or authorization tokens.
2. **Send** an HTTP POST request to the target URL: <https://dev-brisurf.brins.co.id/bmine/CoveringFlagAsuransi>

Proof of Concept:





CONTACT US AT

sec-tsi@work.brins.co.id